

MIP_PROMPT_LIBRARY_V2_PART4_GRAPH.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 4 - Knowledge Graph Prompts

Version: 2.0

Status: Approved

Purpose

The Knowledge Graph is the heart of MIP.

Most modernization tools stop at inventories.

MIP continues.

Inventory

↓

Metadata

↓

Relationships

↓

Knowledge Graph

↓

Reasoning

↓

Modernization Intelligence

The graph enables engineers to ask questions that are impossible to answer manually.

Examples:

- What breaks if I change this field?
 - Which programs update this table?
 - Which batch jobs consume this dataset?
 - What is the true root program?
 - Which components belong together?
-

Graph Design Principles

Everything Is Connected

Programs do not exist in isolation.

Everything is modeled as relationships.

Relationships Are First-Class Citizens

The relationship is often more important than the node.

Explainability

Every graph edge must be traceable to source evidence.

Scalability

Graph must support:

- Millions of nodes
 - Tens of millions of edges
-

Prompt 16

Call Graph Construction

File

prompts/graph/build_call_graph.md

Purpose

Create a complete application call graph.

Context

Program-to-program relationships reveal execution architecture.

Inputs

Program Metadata

Relationship Metadata

Instructions

Create graph:

Nodes:

- Programs

Edges:

- CALLS

Capture:

- Direct Calls
- Recursive Calls
- Circular Dependencies

Generate traversal strategies.

Expected Output

Call Graph Design

Node Model

Edge Model

Traversal Examples

Constraints

Use evidence-based relationships only.

Success Criteria

Every program dependency represented.

Prompt 17

Batch Execution Graph

File

prompts/graph/build_batch_graph.md

Purpose

Model batch execution architecture.

Context

JCL jobs orchestrate business processing.

Inputs

Job Metadata

Program Metadata

Dataset Metadata

Instructions

Create graph.

Nodes:

- Jobs
- Steps
- Programs
- Datasets

Edges:

- EXECUTES
- READS
- WRITES

Generate execution paths.

Expected Output

Batch Graph Architecture

Execution Traversals

Dependency Views

Success Criteria

Batch execution chains become visible.

Prompt 18

Data Lineage Graph

File

prompts/graph/build_data_lineage_graph.md

Purpose

Construct enterprise data lineage.

Context

Data lineage is required for modernization and compliance.

Inputs

Dataset Metadata

DB2 Metadata

Program Metadata

Instructions

Model:

- Producers
- Consumers
- Transformations

Track:

- Source Data
- Intermediate Data
- Target Data

Expected Output

Lineage Model

Lineage Traversals

Impact Analysis Paths

Success Criteria

End-to-end lineage becomes traceable.

Prompt 19

Impact Analysis Graph

File

prompts/graph/impact_analysis.md

Purpose

Determine impact of proposed changes.

Context

Impact analysis is one of the highest-value modernization capabilities.

Inputs

Knowledge Graph

Target Node

Instructions

Analyze:

- Upstream Dependencies
- Downstream Dependencies
- Consumers
- Producers

Generate impact report.

Expected Output

Impact Summary

Affected Components

Risk Assessment

Recommended Actions

Success Criteria

Engineers can estimate change risk before implementation.

Prompt 20

Root Program Detection

File

prompts/graph/root_program_detection.md

Purpose

Identify true entry points.

Context

Many organizations do not know which programs are actual roots.

Inputs

Call Graph

Batch Graph

Transaction Metadata

Instructions

Identify:

- Batch Drivers
- Online Drivers
- Scheduler Entry Points
- Utility Entry Points

Classify roots by type.

Expected Output

Root Program Catalog

Entry Point Graph

Dependency Rankings

Success Criteria

Execution entry points become visible.

Graph Completion Criteria

MIP should now answer:

- Which program calls this program?
- Which programs are impacted?
- Which jobs execute this program?
- Which datasets flow through the system?
- Which tables are affected?
- What are the root programs?
- What is the execution architecture?

At this stage MIP becomes a modernization intelligence platform rather than a repository scanner.

End of Part 4.